端到端加密即时通讯课设最终执行清单

1. 底线检查结论

本清单以 NECESSARY_REQUIREMENT.md 为硬性标准，对当前的 Project_Plan_and_Requirements.md 做了一次“保底达标”检查。

结论：当前规划的大方向是正确的，但如果要严格达到 NECESSARY_REQUIREMENT.md，必须把“最低合规版”单独收敛出来，并补上 6 类不能遗漏的内容。

1.1 对照检查表

硬性要求	当前规划情况	底线判断	必须补强动作
选择或设计一种消息加密算法	已覆盖，采用 RSA + AES-GCM	通过	固定“聊天消息正文必须走 AES-GCM”，不要再把 RSA 直接作为聊天主加密路径
设计一种密钥交换或分发机制	已覆盖，采用 RSA 交换 AES 会话密钥	通过	明确握手流程、密钥字段、异常处理和会话生命周期
即时通信基础功能	已覆盖，但之前规划同时追求桌面端、Web 端、文件传输	需收敛	先定义最低合规版为“桌面端对桌面端一对一文本聊天，发送密文，接收端自动解密显示”
充分代码注释	规划中提到要注释，但未落实到模块级任务	需补强	每个核心模块必须带 docstring、关键流程注释、提交前做一次注释审查
程序可运行且结果正确	规划中提到部署和测试，但未形成保底验收步骤	需补强	新增本地运行步骤、双客户端联调步骤、错误分支测试和演示脚本
原创性要求	规划中未显式落地	需补强	新增设计说明、协议说明、自定义 UI 亮点、自主测试记录，保留你自己的设计决策文档
最终报告固定结构	规划中提到报告，但没有严格绑定 8 个章节	需补强	报告目录必须严格按任务分析、理论基础、设计、实现与测试、结果分析、截图、使用说明、团队分工来组织

硬性要求	当前规划情况	底线判断	必须补强动作
第 8 周课上汇报	规划中提到 PPT， 但不够刚性	需补强	单独准备 15 分钟汇报脚本， 内容必须包括理论、代码、 结果说明
第 9 周最终提交	规划中提到打包， 但未写明邮件和命名	需补强	单独新增提交检查单， 强制核对压缩包内容、邮箱、 命名规则

1.2 底线版最终定义

为了保证课程要求一定达标，本项目的“最低合规版”定义如下：

1. 使用 Python 实现。
2. 使用 AES-GCM 对聊天消息正文进行加密与解密。
3. 使用 RSA-OAEP 对 AES 会话密钥进行交换。
4. 有一个服务端和两个桌面客户端。
5. 客户端 A 可以向客户端 B 发送密文消息。
6. 服务端只转发数据，不解密。
7. 客户端 B 收到后自动解密并显示明文。
8. 核心源码具有充分注释。
9. 程序可以稳定运行并完成演示。
10. 报告、汇报材料和最终提交物完整。

说明：

- Web 端不是 NECESSARY_REQUIREMENT.md 的硬性要求，它属于增强项。
- 文件传输不是 NECESSARY_REQUIREMENT.md 的硬性要求，它属于增强项。
- Wireshark 抓包证明不是硬性要求，但非常适合作为加分项，保留在增强阶段。

2. 固定技术决策

以下技术决策在开发过程中不再反复摇摆，避免任务发散。

2.1 底线版技术选型

- 语言：Python
- 非对称加密：RSA-OAEP(SHA-256)
- 对称加密：AES-GCM
- 通信方式：Python WebSocket 服务端中继
- 桌面客户端：Tkinter 继续演进，单独新建聊天 GUI 文件
- 打包方式：PyInstaller

2.2 关键决策说明

- 继续保留现有 rsa_core.py，不推倒重来。
- 不直接把 InfoSecurWork_GUI.py 改成聊天主程序，避免破坏现有单机演示工具。
- 新建聊天客户端入口文件，现有 RSA GUI 保留为“算法演示工具”。
- 先完成双桌面文本聊天，再做 Web 端、文件传输和抓包展示。

3. 推荐的实际开发结构

为了最小化重构成本，建议先在当前仓库根目录下扩展，而不是一上来大规模迁移目录。

```
InfoSecurityProj/
├─ rsa_core.py                    # 现有，继续复用
├─ InfoSecurWork_GUI.py          # 现有，保留为 RSA 单机演示工具
├─ aes_core.py                   # 新增，AES-GCM 加解密
├─ message_crypto.py             # 新增，RSA + AES 混合加密封装
├─ chat_protocol.py              # 新增，消息协议与序列化
├─ chat_server.py                # 新增，服务端盲转发
├─ chat_client.py                # 新增，客户端网络层
├─ session_manager.py            # 新增，会话、公钥、AES 密钥管理
├─ desktop_chat_gui.py           # 新增，桌面聊天主界面
├─ tests/
│  ├─ test_crypto.py             # 新增，密码模块测试
│  ├─ test_protocol.py           # 新增，协议模块测试
│  └─ manual_acceptance.md       # 新增，人工验收清单
├─ docs/
│  ├─ report_outline.md          # 新增，最终报告结构骨架
│  ├─ result_note.md             # 新增，实验结果说明
│  ├─ design_decisions.md        # 新增，原创性与设计说明
│  ├─ week8_demo_script.md       # 新增，第 8 周汇报脚本
│  ├─ submission_checklist.md    # 新增，最终提交检查单
│  └─ team_division.md           # 新增，可选，小组分工
└─ final_plan.md                 # 本文件
```

4. 可直接开做的详细开发任务清单

以下任务按“必须先做的保底版”和“后做的增强版”拆分。只有前一项完成并验收后，才进入下一项。

4.1 P0 保底达标版

任务 A：整理并改造 rsa_core.py

目标：保留现有 RSA 能力，转为“密钥管理 + 会话密钥交换”的底层模块。

文件：rsa_core.py

必须完成的内容：

- ☐ 保留 RSA 密钥对生成、保存、加载、导出 PEM 的能力。
- ☐ 保留对方公钥导入能力。
- ☐ 保留 encrypt_session_key_for_peer 和 decrypt_session_key_from_peer 这类会话密钥交换接口。
- ☐ 新增公钥指纹函数，例如 get_public_key_fingerprint()，用于界面展示和人工核验。
- ☐ 明确注释：RSA 直接加密长消息仅保留为教学演示，不作为聊天消息主链路。
- ☐ 所有公开方法补全 docstring，说明输入、输出和用途。

完成标准：

- ☐ 能生成本地 RSA 密钥对。
- ☐ 能导出公钥给对方。
- ☐ 能用对方公钥加密 AES 会话密钥。
- ☐ 能用本地私钥解密 AES 会话密钥。

任务 B：新增 AES 消息加密模块

目标：实现聊天消息正文的实际加密算法，满足 NECESSARY_REQUIREMENT 对消息加密算法的要求。

文件：aes_core.py

必须完成的内容：

- ☐ 实现 generate_aes_key()，生成 128 bit 或 256 bit AES 密钥。
- ☐ 实现 encrypt_text(plaintext: str, key: bytes) -> dict。
- ☐ 实现 decrypt_text(cipher_payload: dict, key: bytes) -> str。
- ☐ 使用 AES-GCM，输出至少包含 nonce、ciphertext。
- ☐ 统一 Base64 编码规则，保证网络传输时字段可序列化。
- ☐ 对空消息、错误密钥、篡改数据给出明确异常。
- ☐ 每个关键步骤加注释，说明为何使用 AES-GCM 以及它如何保证机密性和完整性。

建议输出结构：

```
{
  "nonce": "base64_nonce",
  "ciphertext": "base64_ciphertext"
}
```

完成标准：

- ☐ 同一条消息加密后不能直接看到明文。
- ☐ 使用正确密钥可以稳定解密。
- ☐ 篡改密文后解密必须失败。

任务 C：新增混合加密封装模块

目标：把 RSA 和 AES 串成完整链路，生成真正用于聊天消息的加密包。

文件：message_crypto.py

必须完成的内容：

- ☐ 实现 encrypt_chat_message(plaintext, peer_public_key, sender_key_manager)。
- ☐ 内部流程固定为“生成 AES 会话密钥 -> AES 加密正文 -> RSA 加密 AES 密钥 -> 组包返回”。
- ☐ 实现 decrypt_chat_message(message_payload, local_private_key)。
- ☐ 输出统一字典结构，供 chat_protocol.py 直接使用。
- ☐ 增加可视化所需的调试字段生成逻辑，例如原文长度、密文长度、密钥指纹。
- ☐ 给关键步骤写注释，确保之后写报告时能直接解释实现原理。

建议输出结构：

```
{
  "wrapped_key": "base64_wrapped_key",
  "nonce": "base64_nonce",
  "ciphertext": "base64_ciphertext"
}
```

完成标准：

- ☐ 发送方能生成完整加密包。
- ☐ 接收方能用本地私钥和 AES 密钥链路还原原文。

任务 D：新增消息协议模块

目标：把网络消息统一成可序列化、可扩展、可调试的格式。

文件：chat_protocol.py

必须完成的内容：

- ☐ 定义消息类型常量：register、public_key、chat_message、ack、error、heartbeat。
- ☐ 定义统一字段：type、sender_id、receiver_id、timestamp、payload。
- ☐ 实现 make_register_message()。
- ☐ 实现 make_public_key_message()。
- ☐ 实现 make_chat_message()。

- [ ] 实现 parse_message() 和基础字段校验。
- [ ] 每种消息结构写清楚注释，方便后续报告中的“详细设计”章节直接引用。

建议聊天消息结构：

```
{
  "type": "chat_message",
  "sender_id": "alice",
  "receiver_id": "bob",
  "timestamp": "2026-04-15T12:00:00Z",
  "payload": {
    "wrapped_key": "...",
    "nonce": "...",
    "ciphertext": "..."
  }
}
```

完成标准：

- [ ] 所有网络消息都能以 JSON 字符串稳定序列化与反序列化。
- [ ] 非法消息能及时被识别并报错。

任务 E：新增服务端中继模块

目标：实现“只转发，不解密”的服务端，满足即时通信基础功能。

文件：chat_server.py

必须完成的内容：

- [ ] 实现客户端连接接入。
- [ ] 实现用户注册，维护 user_id -> 连接对象 的映射。
- [ ] 实现公钥上传和缓存。
- [ ] 实现消息按 receiver_id 路由转发。
- [ ] 实现心跳保活与掉线清理。
- [ ] 服务端日志只记录连接、消息类型、发送方、接收方、消息长度。
- [ ] 显式保证服务端不解密 payload，不打印明文，不保存私钥。
- [ ] 所有核心函数写注释，说明为什么服务端是盲转发。

完成标准：

- [ ] 两个客户端都能连接到服务端。
- [ ] A 发给 B 的消息可以通过服务端转发到 B。
- [ ] 服务端日志中看不到聊天明文。

任务 F：新增客户端网络层模块

目标：把 GUI 和网络解耦，避免后续界面改动影响通信稳定性。

文件：chat_client.py

必须完成的内容：

- ☐ 实现 connect() 和 disconnect()。
- ☐ 实现 register(user_id, public_key)。
- ☐ 实现 send_chat_message(receiver_id, encrypted_payload)。
- ☐ 实现 receive_loop()，持续监听服务器消息。
- ☐ 设计回调接口给 GUI 使用，例如 on_message、on_error、on_status_change。
- ☐ 增加发送失败、掉线、重连异常处理。
- ☐ 用注释说明网络线程与 GUI 主线程之间的数据流。

完成标准：

- ☐ GUI 不直接操作 socket/WebSocket 底层对象。
- ☐ 收发消息可通过回调更新界面。

任务 G：新增会话管理模块

目标：统一管理本地密钥、对方公钥、会话状态和最近联系人。

文件：session_manager.py

必须完成的内容：

- ☐ 保存本地 RSAService 或 KeyManager 实例。
- ☐ 保存 peer_id -> peer_public_key 的映射。
- ☐ 提供 load_local_keys()、export_local_public_key()。
- ☐ 提供 set_peer_public_key() 和 get_peer_public_key()。
- ☐ 提供 encrypt_for_peer() 和 decrypt_from_peer() 的统一入口。
- ☐ 提供安全状态摘要，例如当前联系人、是否已建立安全通道、公钥指纹。
- ☐ 用注释写清楚会话状态与密钥状态之间的关系。

完成标准：

- ☐ GUI 不直接拼接加密链路，只调用 session_manager。

任务 H：新增桌面聊天主界面

目标：形成真正的课程主程序，满足“即时通信软件基础功能”。

文件：desktop_chat_gui.py

必须完成的内容：

- ☐ 登录区：输入 user_id、服务器地址、连接按钮。
- ☐ 密钥区：生成本地密钥、加载私钥、导出公钥、导入对方公钥。
- ☐ 联系人区：输入对方 user_id 或选择当前联系人。
- ☐ 聊天区：消息列表、输入框、发送按钮。
- ☐ 状态区：显示连接状态、当前联系人、公钥指纹。
- ☐ 收到聊天消息后自动解密并显示明文。
- ☐ 发送前必须先做 AES 加密和 RSA 会话密钥封装。
- ☐ 发送失败、解密失败、未导入对方公钥时弹窗或状态提示。
- ☐ 右侧增加 Crypto Console，打印加密和解密过程日志。
- ☐ 所有 GUI 事件函数写注释，清楚说明触发逻辑。

最小演示流程：

1. 客户端 A 连接服务器。
2. 客户端 B 连接服务器。
3. A/B 各自加载或生成 RSA 密钥。
4. A 导入 B 的公钥。
5. A 向 B 发送一条消息。
6. B 自动解密并在界面显示明文。

完成标准：

- ☐ 双桌面客户端文本聊天链路跑通。
- ☐ 发送链路中界面显示加密日志。
- ☐ 接收链路中界面显示解密日志。

任务 I：新增自动化与人工验收任务

目标：满足“程序可运行且结果正确”的红线要求。

文件：tests/test_crypto.py

必须完成的内容：

- ☐ 测试 RSA 会话密钥加解密。
- ☐ 测试 AES-GCM 正常加解密。
- ☐ 测试篡改密文后解密失败。
- ☐ 测试协议序列化和反序列化。

文件：tests/test_protocol.py

必须完成的内容：

- ☐ 测试 register 消息结构。
- ☐ 测试 chat_message 消息结构。
- ☐ 测试非法字段缺失时报错。

文件：tests/manual_acceptance.md

必须完成的内容：

- ☐ 记录本地启动服务端步骤。
- ☐ 记录启动两个客户端步骤。
- ☐ 记录发送测试消息步骤。
- ☐ 记录异常测试步骤。
- ☐ 记录截图清单。

完成标准：

- ☐ 自动化测试能跑通核心密码功能。
- ☐ 人工测试能完整复现实验演示过程。

4.2 P1 课设增强版

以下内容不是 NECESSARY_REQUIREMENT 的底线要求，但建议在 P0 稳定后继续完成，以提升答辩效果。

任务 J：Web 端客户端

文件：web_client/index.html

必须完成的内容：

- ☐ 输入 user_id、服务器地址、联系人 ID。
- ☐ 显示消息列表。
- ☐ 显示加密控制台简化版。

文件：web_client/app.js

必须完成的内容：

- ☐ 与 chat_protocol.py 保持一致的协议字段。
- ☐ 负责连接 WebSocket、收发消息、更新页面。

文件：web_client/style.css

必须完成的内容：

- ☐ 做出简洁稳定的聊天布局。
- ☐ 兼容平板与手机宽度。

说明：

- Web 端是增强项，不得阻塞 P0 交付。

任务 K：文件传输

文件：file_transfer.py 或整合进 message_crypto.py

必须完成的内容：

- ☐ 限制文件大小。
- ☐ 对文件内容使用 AES-GCM 加密。
- ☐ 文件 AES 密钥使用 RSA 包装。
- ☐ 接收方完成解密后保存文件。

说明：

- 文件传输不是 NECESSARY_REQUIREMENT 的底线项，放在 P1。

任务 L：Wireshark 抓包证明

必须完成的内容：

- ☐ 在本地或公网部署服务端。
- ☐ 抓取聊天过程中的网络载荷。
- ☐ 截图保留密文传输证据。
- ☐ 在报告中与界面明文截图做对照。

任务 M：Windows 打包

必须完成的内容：

- ☐ 编写 PyInstaller 打包命令。
- ☐ 确认生成 exe 可运行。
- ☐ 把运行说明写入 docs/result_note.md 或 README。

5. 文档与提交物任务

这一部分直接对齐 NECESSARY_REQUIREMENT.md 中的“报告结构”和“最终提交物”。

任务 N：固定最终报告结构

文件：docs/report_outline.md

必须严格包含以下章节：

- ☐ 任务分析
- ☐ 相关理论及技术基础
- ☐ 解决思路与设计

- ☐ 实现与测试
- ☐ 任务结果说明与结果分析
- ☐ 运行截图
- ☐ 程序使用说明
- ☐ 团队分工情况（若适用）

要求：

- ☐ 章节标题必须与任务书要求一一对应。
- ☐ 理论基础必须写清网络通信、消息加密算法、密钥交换机制。
- ☐ 设计部分必须拆成概要设计和详细设计。

任务 O：实验结果说明

文件：docs/result_note.md

必须完成的内容：

- ☐ 说明运行环境。
- ☐ 说明如何启动服务端和客户端。
- ☐ 说明测试结果。
- ☐ 说明异常情况及处理。
- ☐ 说明抓包结果或截图结果。

任务 P：原创性证明材料

文件：docs/design_decisions.md

必须完成的内容：

- ☐ 写明为何选择 AES-GCM。
- ☐ 写明为何使用 RSA 交换会话密钥。
- ☐ 写明为何服务端采用盲转发。
- ☐ 写明自己的消息协议设计。
- ☐ 写明自己的 UI 亮点，例如 Crypto Console。

说明：

- 这是“我掌握了代码并做了原创设计”的直接证据。

任务 Q：第 8 周汇报材料

文件：docs/week8_demo_script.md

必须完成的内容：

- ☐ 15 分钟汇报结构。
- ☐ 理论研究要点。

- [ ] 代码展示顺序。
- [ ] 运行结果演示步骤。
- [ ] 关键台词，确保逻辑清楚。

建议 PPT 结构：

- [ ] 课题目标
- [ ] 算法与密钥交换原理
- [ ] 系统架构
- [ ] 核心代码说明
- [ ] 演示结果
- [ ] 总结

任务 R：最终提交检查单

文件：docs/submission_checklist.md

必须完成的内容：

- [ ] 最终报告已完成。
- [ ] 程序源码已整理。
- [ ] 可运行程序或运行说明已准备。
- [ ] 实验结果说明已准备。
- [ ] 压缩包命名符合“第X组+题目”。
- [ ] 若是小组作业，由组长提交。
- [ ] 邮件地址为 xiongf@bjtu.edu.cn。

任务 S：团队分工材料

文件：docs/team_division.md

条件任务：

- [ ] 如果是个人项目，可标注“本项目为个人完成”。
- [ ] 如果是小组项目，必须写清每个人负责的模块、完成情况和交付内容。

6. 注释红线执行规则

为了满足“源码必须有充分注释，否则视为抄袭”的要求，开发期间执行以下规则。

6.1 每个核心文件必须满足

- [ ] 文件顶部有模块作用说明。
- [ ] 每个 public function / method 有 docstring。

- ☐ 非直观逻辑前有解释性注释。
- ☐ 网络、加密、线程、异常处理代码必须说明原因。

6.2 重点必须注释的模块

- ☐ rsa_core.py：密钥生成、密钥交换、指纹生成。
- ☐ aes_core.py：AES-GCM 加密流程和字段说明。
- ☐ message_crypto.py：混合加密完整链路。
- ☐ chat_protocol.py：消息字段意义。
- ☐ chat_server.py：盲转发逻辑和不解密原则。
- ☐ chat_client.py：网络收发与 GUI 交互关系。
- ☐ desktop_chat_gui.py：界面事件和 Crypto Console 更新逻辑。

6.3 提交前注释审查

- ☐ 抽查任意 3 个核心模块，确认只看注释就能理解主流程。
- ☐ 确认没有只有“做了什么”但没有“为什么这样做”的空洞注释。

7. 最终验收门槛

只有下面这些条件全部满足，才能认为项目达到 NECESSARY_REQUIREMENT 的底线标准。

7.1 功能验收

- ☐ 服务端可启动。
- ☐ 两个桌面客户端可连接。
- ☐ 客户端 A 可向客户端 B 发送加密消息。
- ☐ 服务端转发的是密文消息包。
- ☐ 客户端 B 收到后可自动解密展示明文。

7.2 工程验收

- ☐ 核心代码有充分注释。
- ☐ 程序运行步骤清晰。
- ☐ 有测试记录和结果说明。
- ☐ 有报告骨架和截图清单。

7.3 交付验收

- ☐ 形成最终报告。
- ☐ 形成程序源码和可运行版本或运行说明。
- ☐ 形成实验结果说明。
- ☐ 形成汇报材料。

- [ ] 形成最终提交检查单。

8. 实施顺序建议

为了避免返工，建议严格按以下顺序执行：

1. 先改造 rsa_core.py。
2. 再实现 aes_core.py。
3. 再实现 message_crypto.py。
4. 再实现 chat_protocol.py。
5. 再实现 chat_server.py。
6. 再实现 chat_client.py。
7. 再实现 session_manager.py。
8. 再实现 desktop_chat_gui.py。
9. 再补 tests 和 docs。
10. P0 稳定后，再做 Web 端、文件传输、抓包和打包。

9. 最后结论

从 NECESSARY_REQUIREMENT.md 的角度看，最重要的不是一次性把所有增强项做满，而是确保先完成“最低合规版”：

- Python 实现。
- 有消息加密算法。
- 有密钥交换机制。
- 有即时通信基础功能。
- 有充分注释。
- 程序可运行且结果正确。
- 报告、汇报和提交材料完整。

这份 final_plan.md 的作用，就是把原来的大规划压缩成一套真正可以开工的执行表。后续实现时，必须以 P0 为第一优先级，任何增强项都不能阻塞底线交付。